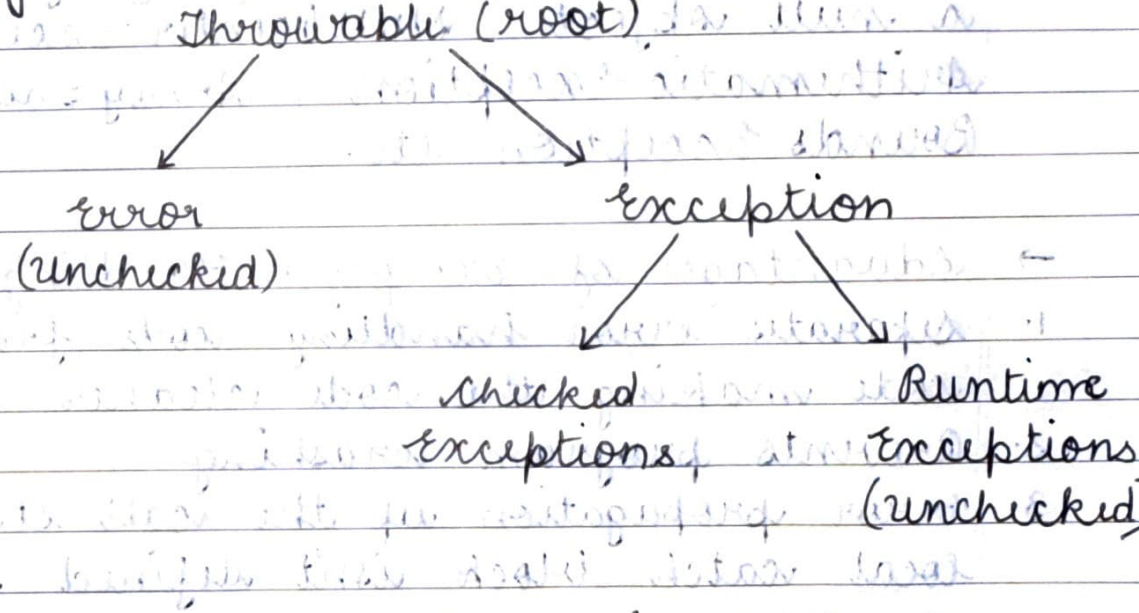
unit → 3

→ exception Handling

- An exception is a unwanted / unexpected event that occurs during the execution of program at runtime and disrupts the normal flow of instructions.
- exception Handling is a powerful mechanism built into java to handle these runtime errors ensuring that the program doesn't crash abruptly & can either recover or fail safely.

→ exception Hierarchy

All exceptions and error classes in java comes from java.lang.Throwable class



1. error
   - error is a problem that typically can't be recovered by application code
   - They are thrown by JVM
   - eg → Out Of Memory error, stack overflow error

2. Checked exceptions (compile time)
- These exceptions are checked by compiler at compile time
- If a method throws a checked exception it must either handle it using a try catch block or declare it in its signature using throws
- eg → IO exception, SQL exception, Class Not Found exception etc.

3. Unchecked exceptions (Runtime)
- exceptions that occur at runtime due to logical flaws or bad data in code
- Compiler doesn't force you to catch them
- eg → dividing by zero, trying to access a null object, Null Pointer exception, Arithmatic exception, Array Index Out of Bounds exception etc.

→ Advantages of exception Handling
1. Separates error handling code from main code making the code cleaner
2. Prevents program crashing
3. error propagation up the call stack if local catch block isn't defined to handle an error in method.
4. Meaningful, improved & faster error reporting & debugging
5. Enforced compile time robustness
6. Safer resource management.

→ Try and catch Block

1. Try

- Try keyword is used to enclose a block of code that is risky - meaning it has the potential to throw an exception during runtime
- A try block must always be followed by either catch / finally or both

2. catch

- It handles the exception thrown by preceding try block. It catches the exception object allowing the program to take corrective action instead of crashing
- There can be multiple catch blocks for a single try block to handle different types of exceptions separately

eg → 1

```
public class Main {
public static void main (String [] args) {
sout ("Start of program");
try {
int result = 10/0;
sout ("Result :" + result);
} catch (Arithematic exception e) {
sout ("caught");
  }
sout ("end of program");
  }
}
```

10/0 throws arithematic exep.

→ gets skipped

↳ object reference variable

eg →2
```
public class Main {
public static void main (String [] args) {
try {
String text = NULL;
sout ("length :" + text.length ());
} catch (Null Pointer Exception e) {
sout ("Tried to access null object");
} catch (Exception e) {
sout ("general exception occured");
}
}
}
```

→ If there are multiple error lines in a try block, as soon as first error is encountered & solved by catch, the JVM comes out of try block and remaining errors are skipped.

→ Finally

- The finally block contains code that always executes regardless whether an exception occured, was caught or if a method returns early.
- Its primarily used for clean up tasks like closing files, database connections or scanners.

```
import java.util.Scanner;
public class Main {
public static void main (String [] args) {
Scanner sc = new Scanner (System.in);
```

```
try {
sout ("enter a number");
int num = sc.nextInt();
int result = 100/num;
sout ("Result :" + result);
} catch (Arithmatic exception e) {
sout ("can't ÷ by 0");
} finally {
sc.close();
sout ("scanner closed by finally block");
}
}
}
```

→ Throw

- The throw keyword is used to explicitly create and throw a exception from a method or any block of code. It is typically used when a program encounters a custom validation failure or illegal argument
- Throw is used inside a method body to explicitly create & throw an exception object
- eg → age is int but user can't enter negative age / can't try to withdraw money more than which they have
- As throw statement is executed, the compiler stops and finds the catch block.

eg :-

```
public class Main {
void validateAge (int age) {
if (age < 18) {
throw new IllegalArgumentException ("Age must be 18 or above to vote");
} else {
sout ("Eligible voter");
}
}
public static void main (String [] args){
try{validateAge (15);
} catch (IllegalArgumentException e){
sout ("caught exception:" + e.getMessage());
}
}
}
```

→ Throws

- It's used in the header of the method while declaring the method
- It doesn't create an exception, it only warns that an exception might fly out of this method
- It can declare multiple comma separated exceptions at a time

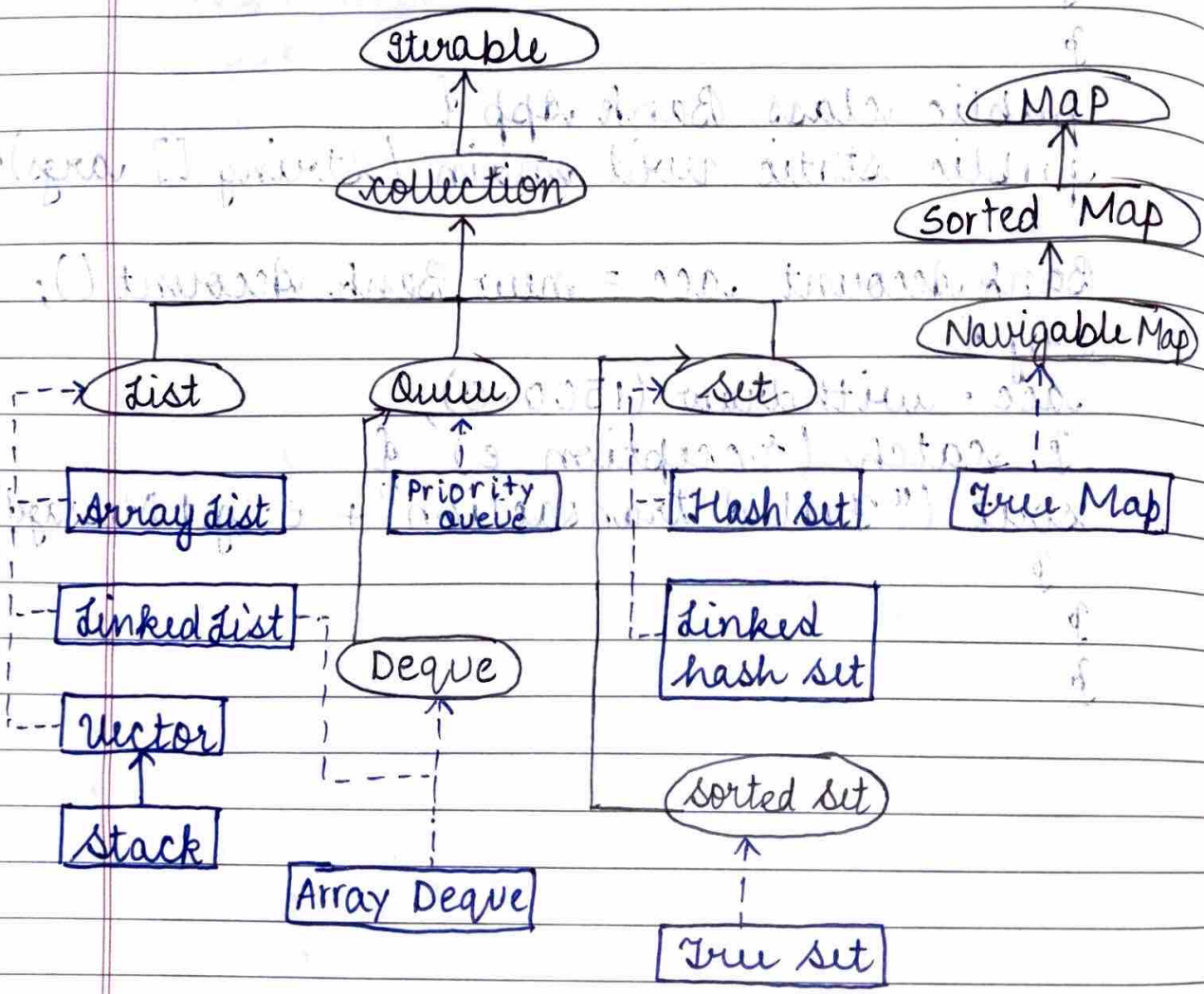
for eg :-

```
class BankAccount {
private double balance = 1000.0;
public void withdraw (double amount)
throws exception {
if (amount > balance) {
throw new exception ("Insufficient Balance");
}
balance -= amount;
sout ("withdrawl successful");
}
}
public class BankApp {
public static void main (String [] args) {

BankAccount acc = new BankAccount ();
try {
acc.withdraw (1500.0);
} catch (exception e) {
sout ("Failed transaction" + e.getMessage());
}
}
}
```

→ JAVA collection Framework

- JCF provides standardized set of interfaces and classes that abstract away the underlying data structures allowing the developers to focus on business logic rather than writing custom data structure implementations.
- JCF reduces the programming effort, increases performance & efficiency, easier reusability & interoperability and increased safety generics.



→ Array List

- Non continous memory
- Variable size
- can only store objects and not primitive like int, float etc.

```
import java.util.Collections;
import java.util.ArrayList;
class ArrayLists {
public static void main (String [] args) {
ArrayList <Integer> list = new ArrayList <Integer>;
```

(Integer → data type)

```
list.add(0);
list.add (2);
list.add (3);
```

} adds the mentioned element to the array list

```
sout (list);
int element = list.get (0);
sout (element)
element = list.get (1);
sout (element);
```

(list → just like array; indexes; get(0) gives 0 & get 1 gives 2)

```
list.add (1, 1);
```

(1 → value to insert; 1 → index on given index)

```
list.set (0, 5);
```

→ sets value = 5 at 0th index

```
list.remove (3);
```

→ deletes element at 3rd index

```
sout (list.size());
```

→ tells size of list

```
Collections.sort (list);
```

Collections → Part of JCF in java.util

→ sorts the array list in ascending order.

→ stacks



FIFO

Push, Pop, Peek.

↳ O(1) ← time complexity.

- can be implemented using array, array list and linked list

```
import java.util.stack;
class Stack {
Stack<Integer> stack = new Stack<>();
stack.push(1);
stack.push(2);
stack.push(5);
sout("Top element:" + stack.peek());
stack.pop();
stack.pop();
}
```

→ Vectors

- Based on dynamic resizing (as soon as a new element comes after being full, the size gets doubled).
- Vectors are similar to array lists but there is a difference
1. Vectors are synchronised (thread safe) meaning multiple threads can modify a vector concurrently without corrupting a data but this adds up extra time.

```
import java.util.Iterator;
import java.util.Vector;
public class Vector {
public static void main(String [] args) {
Vector <String> v1 = new Vector <> ();
v1.add ("A");
v1.add ("B");
v1.add ("C");
sout ("size" + v1.size());
sout (v1.get (1)); → element at 1st index
v1.remove ("B");
v1.contains ("A"); → true if element present

Iterator <String> iterator = v1.iterator();
while (iterator.hasNext()) {
sout (iterator.next());
  }
}
}
```

→ Hash Table

- Present in java.util package & implements the map interface and stores data in key-value pairs
  - key: A unique identifier used to look up associated value
  - value: actual data obj. stored corresponding to key
- It uses a technique called hashing to compute a unique index (hash code) for every key allowing exceptionally fast data retrieval, insertion & deletion
- No null keys / values are allowed
- Elements are arranged according to the

hash values of their keys
- every method in hashtable is synchronized as in vectors

```
import java.util.Hashtable;
.....
public class Hash {
public static void main (String [] args) {
Hashtable <Integer, String> Student Map = new
                                Hashtable <> ();
Student Map.put (101, "Aman");
Student Map.put (106, "Aditi");
Student Map.put (108, "Luv");

Student Map.get (106);
    ↳ value at key = 106

Student Map.containsKey (101);
  ↳ True if 101 key present else false

Student Map.remove (101);
}
}
```

→ Lambda expressions
- It is an essentially anonymous method without a name, without an access modifier and without a return type declaration
- It's just a block of code that can be passed around like normal variable or be given to a method to execute later

(parameters) → { body / statements }

↓ can be empty, single / multiple

↳ actual logic or code that executes.

```
interface calculator {
int oprate (int a, int b);
}
public class Main {
public static void main (String [] args) {
calculator add = (a,b) → a+b;
sout (add.oprate (5,10));
}
}
```

→ Stream

- Stream is like a pipeline through which collection elements pass through
- When elements pass through this pipeline it performs various operations like the sorting & filtering
- useful for bulk processing of data

Collection of Data → Create Stream

↓

Terminal operations like collect, reduce → closes the stream now no more ops can be added & produces output

← Intermediate operations like sort, filter etc

→ Ways to create a stream

1. From collection

```
List<Integer> salaryList = Arrays.asList(3,2,1);
Stream<Integer> streamFromList = salaryList.stream();
```

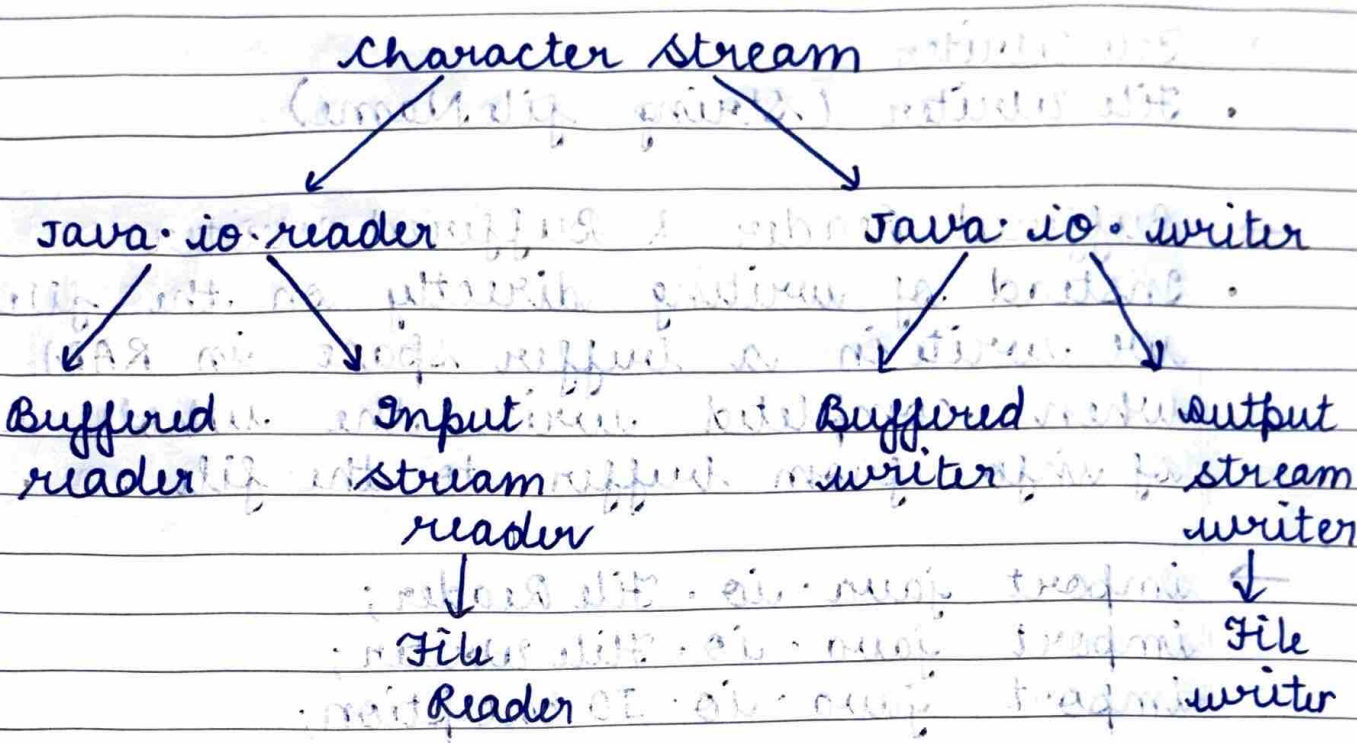
2. From Array

```
Integer[] salaryArray = {3000, 4000, 5000};
Stream<Integer> streamFromArray = Arrays.stream
                                  (salaryArray);
```

3. From static method
4. From stream builder
5. From stream iterate

→ Character Streams

- Used to read and write data (text data)
- Reads / writes (.txt) files
- Uses 16 bit unicode



→ Reader & writer class

- Abstract classes (object can't be created directly)
- Main methods of reader

1. int read () → reads 1 char & returns it's unicode
2. int read (char [] cbuf) → reads char in array
3. void close () → stream closing, frees resources

- Main methods of writer

1. void write (int c): writes 1 char at a time
2. void write (String s): write complete string
3. void flush () : all data of buffer pushed to file
4. void close () : calls flush & closes stream

→ File Reader
- Directly connected with source
uses the constructor
FileReader (String fileName) / FileReader (File file)

→ File Writer
- FileWriter (String fileName)

→ Buffered Reader & Buffered Writer
- Instead of writing directly on the file, first we write in a buffer space in RAM and when completed write the whole chunk of info from buffer to the file

→
```
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

public class Main {
public static void main (String[] args) {
try {
FileWriter fw = new FileWriter ("test.txt");   // → File Name
fw.write ("Hello programmer");
fw.close();                                    // ↳ writes in file

FileReader fr = new FileReader ("test.txt");
int ch;
while ((ch = fr.read()) != -1) {               // if ch = -1 file khtm
System.out.print ((char) ch);                  // sare letter pdhe
}                                              // ↳ (converts ASCII of ch to char)
fr.close();
} catch (IOException e) {
```

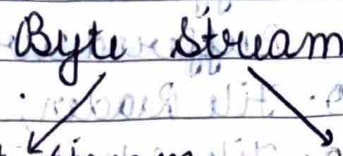
```
sout ("There is some problem" + e.getMessage());
}
}
}
```

→
```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
public class Main {
public static void main (String [] args) {
try {

BufferedWriter bw = new BufferedWriter (
                    new FileWriter ("abc.txt"));
bw.write ("Hello");
bw.newLine (); → to add new line
bw.write ("Meow");
bw.close (); → writes all data from bw
                into file
BufferedReader br = new BufferedReader (
                    new FileReader ("abc.txt");
String line;
while ((line = br.readLine()) != null) {
sout (line);
br.close;
} catch (IOException e) {
 e.printStackTrace ();
}                    └→ JAVA ki error wali line
}
```

→ Byte Streams

- Uses raw bytes (8 bits)
- Used to store binary data like images, audio files, video files, .zip files or non-text files



Java.io.InputStream
- FileInputStream
- BufferedInputStream

Java.io.OutputStream
- FileOutputStream
- BufferedOutputStream

- FileInputStream writes bytes in files and outputstream reads bytes from the file.
- If we use new FileOutputStream ("file.txt") while creating object, purana sara data udd jayega

so we use :-

```
FileOutputStream fos = new FileOutputStream
                       ("file.txt", true);
```

turns on append mode, purana data bhi rahega aur uske peeche new data bhi add hojayega

→

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
public class Main {
public static void main (String [] args){
try{

FileInputStream fis = new FileInputStream
                      ("source.JPG");

FileOutputStream fos = new FileOutputStream
                       ("copy.JPG");
int byteData;
while ((byteData = fis.read()) != -1) {
fos.write (byteData);
  }
fis.close();
fos.close();
sout ("copied from source file to
                copy file");
}catch (IOException e){
 sout (" some error");
 }
}
```

→ File Management using file class

- A file is a collection of data stored on computer typically organized in a specified format and identified by a unique name.
- File class doesn't deal with read / write rather it manages properties, path, size, permissions etc.

File f = new File ("abc.txt");

- METHODS OF FILE CLASS

1. exists () → File exists / not
2. isFile () → checks if its normal file / not
3. isDirectory () → checks if its folder (directory)
4. getName () → gives file / folder name
5. getAbsolutePath () → gives exact path
6. length () → length of file in bytes
7. canRead () → Permission to read / write
8. canWrite () → Permission to read / write
9. createNewFile () → creates new empty file
10. mkdir () → makes new folder
11. mkdirs () → makes new folders + missing parent dir
12. delete () → deletes file
13. renameTo () → changes name / location
14. list () → gives array of all files / folders (names)
15. listFiles () → gives array of all objects of all files and folders of a directory.

→
```
import java.io.File;
import java.io.IOException;
public class Main {
public static void main (String [] args){
try {

File myFile = new File ("abc.txt");

sout ("Name" + myFile.getName());
sout ("Path" + myFile.getAbsolutePath());
sout ("size" + myFile.length());
}
catch (IOException e) {
sout (e.getmessage());
  }
} }
```